Evoltrya OS — Document 2: System Blueprint

What This Document Is

Document 1 laid out, exhaustively, the functional scope, open questions, and business decisions for every module. This Document 2 turns those decisions into system architecture. It no longer asks "whether"; it answers "how the system is built." It defines the overall architecture, the module map, the design principles that run across the whole system, and a precise positioning of the current state. Document 2 in turn drives Document 3 (Roadmap), which gives a phased, executable build plan.

The Nature of Evoltrya OS

One sentence captures the fundamental difference between this system and a conventional ERP:

A conventional ERP starts from accounting documents. Evoltrya OS starts from physical materials.

This is not a slogan; it is the design axiom of the entire system, drawn directly from the fifteen-point analysis of Xero's limitations. In lithium-battery recycling, financial results are generated by physical material movement, production conversion, batch yields, and compliance decisions. Xero can record the final accounting outcome, but it cannot explain how that outcome was produced — through each battery, each pallet, each process step, each permit.

The design mission of Evoltrya OS is to make the full chain from physical material to financial result visible and traceable:

```
Supplier Batch -> Container -> Goods Receipt (GRN) -> Pallet -> Material Category
              -> Work Order -> Output Batch -> Sales Invoice -> General Ledger
```

Every module in the system is a link in this chain. This traceability chain is critical to a recycler's internal control, compliance, production analysis, and external audit — precisely what a general-purpose accounting tool cannot provide natively.

The four foundations

1. **Unified** — procurement, logistics, processing, inventory, sales, compliance, HR, finance, and business development are integrated in a single database, not stitched together from third-party add-ons. Fragmented data is a risk for recycling: one transaction can simultaneously affect inventory, production, compliance, tax, sales, cost allocation, and audit evidence.
2. **Extensible** — the number of modules is not fixed. New modules, whether core or auxiliary, are added continuously under a common module template.

3. **Permission-segregated** — each user accesses only what their permissions allow. The current model is module-level permission (can/cannot enter a module) plus a department/function ownership dimension.
4. **Multi-entity** — data is partitioned by Singapore/EU entity. The two entities are independent P&L entities that can each keep their own books and also be consolidated for group analysis.

A core design constraint: sufficient, not excessive

The system follows a pragmatic engineering principle — reserve the interface for the future, but build only what the present requires. Representative examples:

- The processing module currently implements a single-stage model (one input batch to multiple outputs), but the architecture reserves room to upgrade to a multi-operation model (dismantle -> crush -> pyrolysis -> hydrometallurgy, each operation recorded independently).
- Permission implementation is deferred to a later phase, but from the outset the data layer carries the ownership dimension and RLS hooks, and the user-role table structure is established early.
- The task module's interface currently handles personal tasks only, but the data structure already reserves collaboration/visibility/permission fields, ready to activate with zero rework once the user system is built.

This principle runs across the system: get the load-bearing structure and the standard right first, then fit out the rooms one at a time.

Design Philosophy and Core Principles

These ten principles are not abstract ideals. They are concrete rules already embodied in the code, and they will constrain every future module.

Principle 1: The batch is the unit of accounting

Everything in the system is organised around the batch. A batch of spent batteries is traced as a single unit from inbound, through processing, to output, and on to sale. The batch is the bridge between physical material and financial cost — and batch-level COGS is the recycling industry's most critical capability, the one most absent from Xero.

Principle 2: Traceable end to end, with no breaks

The system must answer every question an audit or management review can pose: which supplier this material came from, which container it arrived in, which pallet it was stored on, whether it was soaked, whether it had voltage, whether it was suitable for repurposing or recycling, which line processed it, which output batch it produced, and who it was sold to. Every link in the chain is structured data, not text buried in an attachment or a note.

Principle 3: Rules are enforced at the database layer

Critical business rules — quantity conservation, state transitions, atomic commits — are enforced at the database layer through triggers and atomic functions, not by front-end validation. The front end can be bypassed; the database cannot. The processing module's atomic commit_processing_run / rollback_processing_run functions embody this principle: a commit or rollback either fully succeeds or is fully reverted, leaving no intermediate state.

Principle 4: Metal content drives pricing, not weight alone

The value of recycled material is not determined by weight or purchase price alone. It also depends on metal content, moisture, impurity level, chemistry, recoverability, and market metal prices. Two ten-tonne batches of black mass can differ greatly in value if one has higher cobalt and nickel content. The pricing model must be able to peg to assay results and metal-market quotes (LME/SMM) — raw material priced as "nickel-cobalt content x LME metal price." This is a capability standard ERP lacks and one that requires dedicated design.

Principle 5: Business data is native; finance is auto-generated

Xero's pain point is that business and finance are disconnected and require duplicate entry. Evoltrya OS's advantage is precisely that business data is native to the system: procurement generates payables, sales generate receivables, stock movements generate inventory journals — business documents auto-generate accounting entries rather than being re-keyed.

Principle 6: Every value-bearing action requires approval

Every value-bearing action in the system (purchase orders, payments, price changes, stock issue, expense claims) passes through an approval chain. Changes to system architecture are made only by the system administrator; updates to business parameters (such as pricing) are made by the functional leader. Approval is two-level: the requester raises the request and the supervisor approves; above a threshold (e.g. 10k) it escalates to CFO approval.

Principle 7: Soft delete, never hard delete

All business records are soft-deleted with deleted_at, recoverable and retained for audit. The system performs no physical deletion — the compliance and audit requirements of recycling demand that data remain traceable and recoverable.

Principle 8: Bilingual parity, enforced at build time

The system is fully bilingual (EN/ZH), with English/Chinese key parity enforced at build time via satisfies Messages — a missing or mismatched translation key fails the build. Stored canonical values are separated from displayed translated labels: data stays canonical, only the interface label is translated.

Principle 9: Operational audit, not only accounting audit

Recycling needs more than an accounting audit trail; it needs an operational audit trail: who received the material, who weighed it, who classified it, who changed the material category, who approved the work order, who recorded the output weight, who approved the production loss, who released the batch for sale, who uploaded the compliance documents, who changed the valuation assumption. The system tracks all of these operational actions within a single material lifecycle.

Principle 10: A unified module template

Every module follows a unified structure to ensure consistency and maintainability: list page (filter/sort/search/paginate/export), detail page, create/edit form (validation), soft delete, state machine (where applicable), automatic coding, bilingual i18n, ownership fields, and operation log. The existing supplier/processing modules demonstrate this standard, and new modules are built by replicating it.

System Architecture

Technology stack

Evoltrya OS is built on a modern, type-safe, full-stack integrated stack:

Layer	Technology	Note
Front-end framework	Next.js 16 (Turbopack) + TypeScript	Server-components first; type safety across the stack
Styling	Tailwind CSS	Unified design language
Database	Supabase (PostgreSQL)	Singapore region; business rules enforced at the DB layer
Auth and permissions	Supabase Auth + RLS	Row-level security, ownership-dimension filtering
File storage	Supabase Storage (private buckets)	Attachments and compliance files, signed-URL access
Deployment	Vercel	Push to main auto-deploys
Internationalisation	In-house lightweight i18n (EN/ZH)	Build-time bilingual parity

The reason for this stack: business data, files, authentication, and permissions are unified in one Supabase database, avoiding the fragmentation risk of a Xero-plus-plugins architecture (the "Unified" foundation).

Layered architecture

The system uses a clean separation of layers, each with a single responsibility:

```
+-----------------------------------------+
|  UI layer                               |
|  Server components (data fetch)         |
|  + client components (interaction)      |
|  Unified module template: list/detail/form|
+-----------------------------------------+
|  Application logic layer                 |
|  (Server Actions / Route Handlers)       |
|  Validation, orchestration, shared queries |
+-----------------------------------------+
|  Database layer (PostgreSQL)             |
|  * The final enforcement point for rules * |
|  Triggers (auto-coding, updated_at)      |
|  Atomic functions (commit/rollback)      |
|  RLS policies (permission, ownership)    |
|  Foreign keys (structural guarantee of the |
|  traceability chain)                     |
+-----------------------------------------+
```

Key design: business rules sink to the database layer. Front-end validation exists for experience; database constraints are the guardian of truth. Quantity conservation, state transitions, atomic commits — these are enforced at the database layer, and no operation that bypasses the front end can compromise data integrity.

Core data flow: the life of a material

The system's main data flow tracks a batch of material through its complete lifecycle, from entry to sale. This flow is also the primary coupling line between modules:

```
Procurement    Logistics/Inbound   Processing      Output/Inventory   Sales
    |               |                   |                 |             |
[Requisition]  [Goods Receipt]   [Input batch]     [Output batch xN] [Sales order]
    |          [Weigh/ticket]    [Work order]  ->  [Black mass/Cu/   [Shipment]
[PO]        -> [Inbound batch]-> [1-to-many         Al/plastic/    -> [Invoice/
[Metal          [Assay]          conversion]        residue]           collection]
 content]       [Batch-traceable][Yield/loss]      [Metal-content     [Metal pricing]
    |               |            [Batch COGS]        valuation]
    v               v                   v                 v             v
   Each step writes an operation log, updates inventory,
   generates a finance journal, and retains compliance documents
                           |
```

```
                          v
              Traceability chain + Batch COGS
```

One-to-many conversion is the defining feature of this flow, and the essence of what separates recycling from manufacturing and ordinary trading: one batch of spent batteries, put into processing, yields black mass, copper, aluminium, plastics, and residue — multiple materials. This one-to-many conversion drives the design of four modules: processing, inventory, cost, and sales (see the processing-module redesign in a later section).

Current technical conventions (shared by all modules)

The following conventions are established in the code and constrain the build of every future module:

- **Automatic coding** — each record type gets a human-readable code from a database trigger (SUP-/CUS-/MAT-/IN-/OUT-/PROC-, in the format PREFIX-YEAR-SEQUENCE).
- **Shared updated_at trigger** — all tables reuse a single update_updated_at() function, never redefined.
- **RLS authenticated-full-access** — the current policy is "authenticated users have full access" (reserved for future fine-grained permissions; in the single-user phase, you are that authenticated user).
- **Soft delete** — every table carries deleted_at, filtered by default in queries.
- **Shared query logic** — a list's search/filter/sort/pagination logic is extracted into a module-level shared helper (e.g. supplierQuery.ts); the list page and the export route reuse the same one and never drift.
- **URL-driven list state** — search/filter/sort/pagination all go through URL parameters: shareable, bookmarkable, preserved on refresh.
- **Private storage buckets + signed URLs** — attachments live in private Supabase Storage buckets and are accessed through time-limited signed URLs, keeping sensitive files (qualifications, compliance) secure.

Cross-Cutting Foundations

These are not individual modules but underlying mechanisms shared by every module. Their design constrains how each module is built, so they are the foundation of the system.

Permission System (RBAC)

Role: Controls who can do what, to which data, in which module. The basis of system-wide segregation.

Established direction: Full permission implementation is deferred to a later phase (unified implementation once module functionality is built out), but the interface is reserved from the outset — the data layer carries the ownership dimension and RLS hooks, and the user-role table structure is established early.

Decisions made:

- **Permission granularity: module level** — controls whether a user can enter a given module. (Operation-level, field-level, and data-scope granularity were not selected, keeping the system simple for its current phase; these can be upgraded later.)
- **Data ownership dimension: by department/function** — each business record carries an owning department/function (rather than ownership by entity or by business line). This determines the ownership fields every table must carry.
- **Login methods:** email + password (current) plus Google/Microsoft SSO; MFA not mandatory for now.

Pre-defined roles (10): Super Admin/Owner, Procurement Officer, Warehouse/Logistics Officer, Processing/Production Supervisor, Sales Representative, Finance, Compliance Officer, HR, Business Development, Read-only/Auditor. Each role maps to a set of accessible modules.

Ownership fields (reserved now): each business record carries owning entity (SG/EU), owning department/function, creator, responsible party. Supabase RLS hooks filter visible data by the user's entity/role.

Multi-Entity Architecture (Singapore + EU)

Role: Two legal entities, with data both independent and able to interoperate.

Decisions made:

- **Entity relationship: two independent P&L entities** — each keeps its own books, and both can be pulled and consolidated for group analysis.
- **Division of business:** the SG entity primarily carries headquarters functions plus non-OECD battery material; the EU entity primarily processes OECD battery material. There is material transfer between the two entities.
- **EU registration country (candidates for the start):** Poland, Italy, France. The final registration country determines tax-ID format, VAT rate, and local hazardous-waste regulations.
- **Currency strategy:** business-related transactions are all in USD; employee compensation and administrative expenses are in local currency (SGD for SG, EUR for EU).
- **Cross-border consideration:** EU cross-border transactions involve intra-EU VAT rules (VAT treatment between member states differs from China-EU / EU-third-country).

Functional scope: entity master data (registration, tax IDs, bank accounts, functional currency), data segregated by entity (only the user's entity visible by default; authorised users may cross entities), inter-entity material transfers, inter-entity internal transactions / transfer pricing (tax implications), consolidated reporting.

Audit and Traceability

Role: Full visibility into who changed what, and when. Especially critical given the recycling industry plus compliance requirements.

The system distinguishes two kinds of audit; recycling needs both.

Accounting audit (who changed what in the books) — present in all systems.

Operational audit (who did what to the material) — specific to recycling and absent from general systems. Within a single material lifecycle, the system tracks: who received the material, who weighed it, who classified it, who changed the material category, who approved the work order, who recorded the output weight, who approved the production loss, who released the batch for sale, who uploaded the compliance documents, who changed the valuation assumption.

Functional scope: operation log (all create/update/delete: operator, time, before/after values), document status-change history (e.g. the full progression of a processing run from draft to submitted), soft deletes (recoverable plus audit retention), full batch traceability (the complete inbound -> processing -> output -> sale chain), audit-report export for key operations (for regulatory inspection).

Compliance retention: the retention period for hazardous-waste records follows local regulation (currently taking Singapore as the reference).

Coding System

Role: Unified numbering rules for documents and master data system-wide.

Decision made: codes reflect entity/year/module. The existing automatic codes (SUP-/CUS-/MAT-/IN-/OUT-/PROC-) will be extended; new modules (purchase order PO, sales order SO, invoice, payment, transfer note) follow the same system and carry an entity identifier (e.g. SG-PO-2026-0001 vs EU-PO-2026-0001).

Common Module Standard (Module Template)

Role: A unified structure followed by every module, ensuring consistency and maintainability. The existing supplier/processing modules already demonstrate it.

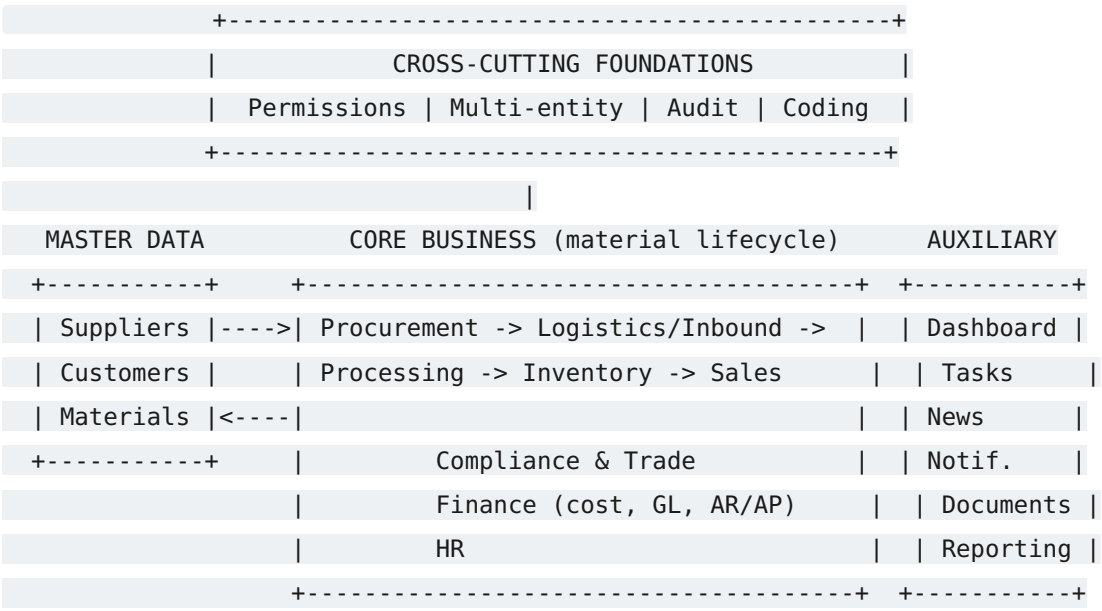
Standard per module: list page (filter/sort/search/paginate/export), detail page, create/edit form (validation), soft delete, state machine (where applicable), automatic coding, bilingual i18n, ownership fields (entity/department/creator), operation log.

Extended common capabilities (recently landed on master-data and transaction modules): attachment management (private bucket + file-type restriction + signed-URL download), CSV export (UTF-8 BOM for correct Chinese rendering), URL-driven search/filter/sort/pagination. These capabilities are now part of the module standard, validated and replicated across the supplier, customer, material, inbound, and output modules.

Module Map

The system comprises master-data modules, core business modules, and auxiliary modules, all coupled along the material lifecycle. The map below states each module's role, current state, and target.

Relationships at a glance



Master data modules

Module	Current state	Target
Suppliers	Full-featured: basic fields, 8-state machine, compliance sub-table, search/filter/sort/pagination, export, attachments	Enrich: classification, rating/evaluation, supplier audit process, contact management, bank accounts, price file, transaction history
Customers	Full list features (search/sort/ pagination/export/attachments); no state machine	Enrich: classification, credit management, contacts/ shipping addresses, bank/invoicing details, transaction history/AR, contract management
Materials	Category/chemistry/unit + free-text fallback; full list features + attachments	Enrich: specifications/technical parameters, multi-level classification tree, unit conversions (kg/tonne), per-material inspection standards, safety stock, material relationships

Core business modules

Module	Current state	Target
Procurement	Not built (inbound is a standalone batch record)	Full cycle: requisition -> approval -> PO -> confirmation -> goods receipt -> inspection -> put-away -> invoice matching -> payment; metal-content pricing; payment management
Logistics	Partial (inbound/output are batch records only)	Transport management, warehouse/bin management, stock in/out documents, packaging/labelling, cross-border logistics
Processing	Skeleton: input/output legs, quantity conservation, atomic commit/rollback	Redesigned around one-to-many recycling conversion (see next section)
Inventory	Read-only material balance	Real-time multi-dimensional stock ledger, batch/shelf-life, stocktake, alerts, metal-content valuation
Sales	Output has an optional customer field; no sales process	Full cycle: inquiry/quote -> SO -> approval -> shipment -> invoicing -> collection; metal-based pricing; multiple pricing models
Compliance & Trade	Not built (supplier compliance sub-table exists)	The industry-core module: qualification/licence management, cross-border transfer (Basel), import/export customs, regulatory filing, regulation library
Finance	Not built	Accounting on par with Xero, solving the fifteen Xero limitations: GL, AR/AP, treasury, batch-level cost accounting, tax, reporting
HR	Not built	Employee master data, attendance, compensation; execution outsourced, data integrated; compliant with local (Singapore) requirements
Business Development	Not built	Opportunity/lead management, cooperation projects, contact/relationship management, conversion to master data

Auxiliary modules

Module	Current state	Target
Tasks/To-Do	Full-featured: kanban, drag-and-drop, CRUD, bilingual; collaboration fields reserved	Team assignment/collaboration (activates with user system), business-module linkage (e.g. "this PO awaits your approval" becomes a task)
Dashboard/ Workbench	Not built	Personal workbench, operating dashboard, role-tailored home page, KPI visualisation
Industry News	Not built	Metal-quote and policy aggregation, daily digest, linkage to pricing
Notification Center	Not built	System notifications (approvals, alerts, expiries), multi-channel, preferences
Document Management	Per-module attachments in place	Cross-module document classification, search, version management, access control
Reporting Center	Per-module exports in place	Cross-module reports, custom reports/queries, scheduled reports

Focus Module Redesign

Two modules warrant deeper design here, because they are where Evoltrya OS most sharply diverges from a generic ERP: Processing (the operational heart) and Finance/Compliance (the reason for building at all).

Processing / Production — Redesigned

Document 1 flagged this module for a complete redesign. The flow originally sketched suited "building up a product" — many parts converging into one finished good, as in manufacturing. Recycling is the opposite: it reduces a product — one batch of spent batteries dispersing into many output materials. The redesign below is built around that one-to-many reality.

Current state: the module records processing runs (input legs / output legs), enforces quantity conservation, provides a traceability chain, and commits/rolls back atomically via RPCs. This is a sound skeleton, but it captures only a basic material balance, not production management.

The core model: one-to-many conversion. A single input batch (or several) yields several distinct output materials. For example, one batch of used lithium-ion batteries may produce black mass, copper, aluminium, plastics, and residue. The design must natively answer:

- How much black mass was produced from this battery batch?
- What was the yield percentage?
- How much copper and aluminium were recovered?

- What was the production loss?
- Which output product should carry which portion of the input cost?

Staging model: single-stage now, multi-operation reserved. The system currently implements a single-stage model — one processing step converts inputs to outputs — which matches present operations. The architecture, however, reserves room for a future multi-operation model, where a run decomposes into sequential operations (dismantle -> crush -> pyrolysis -> hydrometallurgy), each recording its own input, output, loss, and labour, with intermediate products carried between stages. The data structure is designed so that adding stages later is an extension, not a rewrite: the run remains the top-level unit, and an "operation" layer can be inserted beneath it without disturbing the existing input/output leg structure.

Functional scope of the redesigned module:

- **Process and recipe** — routing (which operations turn a material into output), processing recipe/BOM (standard input-output ratios, standard loss rate). Reserved for the multi-operation upgrade.
- **Production planning and execution** — production plan / work order (what to process, how much, when), work-order scheduling, actual-execution record (actual input/output vs plan).
- **Yield and recovery analysis** — loss-rate analysis (with trend/anomaly detection), yield analysis, and metal recovery rate (input metal vs output metal) — a core industry KPI. Not merely weight conservation but metal-element conservation.
- **Cost roll-up (batch COGS)** — processing-cost accounting that apportions raw material plus labour, energy, equipment depreciation, and consumables to each output batch. This is the upgrade from "logging" to "cost management," and it is where a recycler most needs what Xero cannot do.

The cost-allocation question (a key design decision). When one input's cost must be spread across multiple outputs (black mass, copper, aluminium, residue), the system needs a defined allocation basis. Candidate bases include allocation by weight, by metal value, or by market value of each output. The redesigned module makes the allocation basis an explicit, configurable choice rather than an implicit assumption — because the answer directly determines the reported gross margin of each output batch. Quality management (in-process inspection, non-conforming handling) is out of scope per the current requirement, keeping the redesign focused on the material and cost model.

Why this matters. Without this redesign, the company knows its overall profit and loss but not the true gross margin of each batch of black mass or each production run. The redesigned processing module is the point at which the traceability chain acquires its cost dimension — the link that lets every downstream dollar be explained by the input battery, batch, process, and yield that produced it.

Finance and Compliance — Design Anchored in Xero's Limitations

The finance and compliance modules are not designed against a generic accounting checklist. They are designed against a specific, detailed analysis of what Xero cannot do for a battery-recycling business — the fifteen-point limitation study that accompanies this planning set. That study is the design soul of these two modules. The headline gaps the modules must close:

1. **Operations-driven, not accounting-driven** — Xero records financial outcomes; the system must explain how each outcome was created through material movement, conversion, yield, and permits.

2. **Raw materials and work-in-progress** — recycling is not buy-and-sell; it requires raw-material tracking, WIP, production orders, yield and loss calculation, and output-batch creation, which Xero's native inventory does not model.
3. **Batch, pallet, and container traceability** — the full Supplier Batch -> Container -> GRN -> Pallet -> SKU -> Work Order -> Output Batch -> Sales Invoice -> GL chain.
4. **One-to-many production conversion** — allocating one input's cost across multiple outputs and by-products.
5. **Accurate batch-level COGS** — allocating purchase, freight, handling, sorting labour, discharging, electricity, gas/nitrogen, crushing/drying, production labour, consumables, waste treatment, overhead, losses, and by-product offsets to each batch, output product, and yield.
6. **Metal-content-based inventory valuation** — linking valuation to assay results, metal percentage, moisture, impurity, and live market prices, not weight alone.
7. **Repurpose vs recycle classification** — structured batch/pallet-level treatment-path decisions, not free-text notes.
8. **Disposal-fee business models** — a single batch may carry disposal income, inventory value, processing cost, and later sales revenue simultaneously.
9. **Deeply integrated compliance data** — HS code, Basel classification, import/export permits, NEA/DOE approvals, UN number, MSDS, packing declaration, country of origin/destination, container/seal number, soaked/voltage status, treated as structured operational data that gates transactions, not as loose attachments.
10. **Physical-to-financial linkage** — the complete chain that lets an auditor see how physical stock movements support the accounting numbers.
11. **Operational audit trail** — beyond the accounting audit trail (Principle 9).
12. **Production performance analysis** — yield by battery type / supplier / line, recovery rates, cost per tonne, gross margin by work order / supplier / product.
13. **No data fragmentation** — one unified database instead of Xero plus fragmented third-party add-ons.
14. **Multi-country, multi-entity recycling** — shipment ownership transfer, intercompany inventory movement, transit status, permit status by country, cross-border cost allocation, entity- and group-level visibility.
15. **Management decision support** — combining financial, operational, technical, and compliance data to decide which supplier yields best, which chemistry is most profitable, which batch to process or hold.

Finance-to-business integration (the design decision). The direction is that business documents auto-generate journals — a purchase generates a payable journal, a sale a receivable journal, stock movements an inventory journal — exploiting the fact that business data is native to the system (Principle 5). This is the structural answer to Xero's central weakness: business and finance being disconnected and requiring duplicate entry.

Relationship to Xero. The finance module's positioning is to be the operations-and-cost engine that Xero cannot be, with the Xero-vs-integrate boundary decided as the finance module matures. The

immediate design priority is batch-level cost accounting and metal-content valuation — the numbers the business most needs to get straight and that Xero structurally cannot produce.

Compliance as workflow, not documentation. The compliance module treats compliance fields as structured operational data that can gate a shipment, batch, or sales transaction before it proceeds — validating that the required documents (Basel, permits, declarations) are present. Compliance is part of the operating workflow, not an afterthought of attachments.

Precise Current-State Positioning

An honest assessment of where the system stands. This positioning is what Document 3 (Roadmap) plans forward from.

What is built and solid — the framework foundation

- **Authentication and RLS** — Supabase Auth with row-level security, hardened; the ownership dimension and RLS hooks reserved for the permission system.
- **Bilingual i18n (EN/ZH)** — build-time parity enforcement via satisfies Messages; value/label separation.
- **Atomic database transaction pattern** — commit/rollback RPCs with stable, parameterised error codes.
- **Soft deletes, automatic coding, updated_at triggers** — established as shared conventions across all tables.
- **A validated module-development template** — proven twice over: first on the master-data modules, then replicated onto the transaction modules, confirming the template is genuinely reusable and adaptable.

What is built at feature-complete list level — the master data and transaction lists

The following modules now carry the full list-feature standard (URL-driven search, filter, sort, pagination, CSV export with correct Chinese rendering, and — where relevant — private-bucket attachments with file-type restriction):

- **Suppliers** — plus the 8-state machine, compliance sub-table, and attachments. The reference implementation.
- **Customers** — full list features and attachments (status filter deliberately omitted, as status is a single dead value pending a real lifecycle).
- **Materials** — full list features, a category filter dropdown driven by the options list, and attachments with material-specific document categories (spec sheet / MSDS / COA).
- **Inbound** — full list features including cross-table search (by batch code, material name, and supplier name), a stage filter, and supplier/material dropdown filters.

- **Output** — the same, adapted (customer instead of supplier, state instead of stage, with correct handling of the nullable customer for unsold stock).

What is skeleton or empty — the business depth

- **Processing** — a sound skeleton (input/output legs, conservation, atomic commit/rollback) awaiting the one-to-many redesign above.
- **Inventory** — a read-only material balance awaiting the full stock-ledger redesign.
- **Procurement, Sales, Compliance/Trade, Finance, HR, Business Development** — not yet built; all planned.
- **Auxiliary** — Tasks is full-featured; Dashboard, News, Notification Center, and Reporting Center are not yet built.

The accurate metaphor

The system today is a foundation, a load-bearing structure, and a set of finished show units. The master-data and transaction-list modules are the finished show units — they demonstrate, at production quality, the standard to which every module will be built. But the dozens of actual rooms — full business functionality in processing, inventory, procurement, sales, compliance, and finance — remain shell or empty. The value of the work done so far is that the construction method is now proven: from establishing database conventions, to generating and reviewing SQL, to building the UI in verified steps. Every remaining module is built by applying this proven method.

Key Technical Challenges and Design Decisions

The distinctive challenges of a recycling ERP, and the design stance the system takes on each.

Metal-content-based pricing

Raw material is priced by metal content and assay results (e.g. nickel-cobalt content x LME metal price), integrating metal-market quotes (LME/SMM) as a reference. Output is likewise priced against metal markets. This demands a pricing engine that reads assay data and live market prices — a capability standard ERP lacks. The design stance: pricing models are first-class, configurable objects integrated into the sales and procurement modules, not hard-coded fixed prices. Multiple pricing models coexist and are selected per transaction or contract.

Batch-level COGS and cost allocation

The cost of goods sold in recycling spans purchase cost, freight, local transport, unloading/handling, sorting labour, discharging, electricity, gas/nitrogen, crushing/drying, production labour, consumables, waste treatment, overhead allocation, production losses, and by-product offsets. The design stance: cost is accumulated at the batch and work-order level and allocated to output products on an explicit,

configurable basis (by weight, by metal value, or by market value). This makes true per-batch gross margin computable — the number the business most wants and that Xero structurally cannot produce.

One-to-many production conversion

One input batch yields multiple output materials plus by-products and losses. The design stance: the processing module is built around this natively (see the redesign), with quantity conservation enforced at the database layer and cost allocated across outputs. The single-stage model is implemented now; the multi-operation model is reserved as a clean extension.

Two-stage pricing by assay

Raw material may be received at an estimated price and adjusted after assay (price adjustment if actual content is below agreement). The design stance: the procurement and inspection flow supports a receive-then-adjust pattern, with the assay result feeding a price adjustment against the original batch — and the payment schedule reflecting this (a payment-management model such as deposit 80% / arrival 10% / post-assay 10%).

Cross-border compliance and multi-entity operations

Material moves between SG and EU entities, across borders, under Basel and local hazardous-waste regulation, with permits varying by country. The design stance: compliance data is structured and gates transactions; multi-entity operations (transfer, transfer pricing, consolidated visibility) are first-class; and the two entities are independent P&L units consolidatable for group analysis. Business transactions are recorded in USD; compensation and administration in local currency.

Inventory accuracy

The most-cited pain in inventory is accuracy — the alignment of book and physical stock across multiple statuses (available / in-inspection / in-transit / frozen), with aging stock managed and alerted, valued on a moving weighted-average basis, and reconciled through a yearly full stocktake. The design stance: a real-time, multi-dimensional stock ledger where every movement is a traceable line, reconciled to the physical count.

From Blueprint to Roadmap

This blueprint fixes the architecture, the design principles, the module map, and the precise current-state positioning. Document 3 (Roadmap) plans forward from it — a phased, executable build plan with priorities and milestones, sequenced by the module rollout priority established in Document 1: manufacturing (processing) first, then warehousing (inventory/logistics), then finance, then procurement and sales, with the auxiliary and cross-cutting modules woven in. The permission system is implemented in a unified pass once module functionality is built out, its interface having been reserved from the start.

The enduring value of this document is that it translates the large goal of building a complete, industry-specific enterprise OS into a coherent architecture and a set of load-bearing decisions — each traceable back to a real business or domain choice, and each carried forward into an executable plan.